Audit Report

Own Protocol v2

Date: **November 7 — November 9, 2026**

Introduction

eUSD is an overcollateralized stablecoin issued against tokenized real-world assets. Each collateral risk group has its own manager contract, and all managers mint the same eUSD. This review covers the EUSDLiquidityManager, which mints eUSD against concentrated-liquidity positions in asset/eUSD pools. Each position lives in its own vault (a beacon proxy of a DEX adapter: Fables, a Uniswap v4 fork, or Aerodrome Slipstream). The manager values a position at the oracle price, which is a lower bound for a concentrated-liquidity range at any pool price. It charges a stability fee that depends on whether the liquidity is staked in the DEX gauge. Positions can be liquidated in part or in full. Any eUSD holder can redeem eUSD against positions whose vault is mostly asset at the pool price.

The review ran in rounds. Each round used independent multi-agent passes over the source. Every finding was re-verified against the code and, for the DEX adapters, against the live Robinhood Chain contracts. Fixes were re-audited. Proofs of concept ran on a Robinhood Chain fork or as unit tests.

Two final full reviews of all files in scope ran independently on `aba9e7e`, after the last fixes. Neither raised a new issue. Every point they raised was already in this report as acknowledged (for example I-06 on partial liquidation below 100%), or was checked earlier against the live Robinhood Chain contracts and rejected.

Scope

Repository: `own-protocol/own-v2`

Audited Commit: `c4042f53315915478673b399bf3b3c697149c8ff`

Re-audit Commit: `aba9e7ea36c551e744a2422039cd1463b32e1f2f`

Files:

- `src/eusd/EUSDLiquidityManager.sol`
- `src/eusd/adapters/FablesAdapter.sol`
- `src/eusd/adapters/AerodromeAdapter.sol`
- `src/libraries/LiquidityMath.sol`
- `src/interfaces/IEUSDLiquidityManager.sol`
- `src/interfaces/ILiquidityVault.sol`
- `src/interfaces/external/IFables.sol`
- `src/interfaces/external/IAerodrome.sol`

The contracts in scope are deployed on Robinhood Chain from the re-audit commit; addresses are in `docs/contracts-robinhood.md`. The oracle (`ChainlinkOracleVerifier`), `EUSD` and `EUSDManager` were read as context and are out of scope.

Findings

Each issue has an assigned severity:

Critical/High issues are directly exploitable security vulnerabilities that can lead to loss of funds and need to be fixed.

Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.

Low/Info issues are non-exploitable or informational findings that do not pose a direct security risk to the system's integrity.

Issues marked RESOLVED are fixed in the codebase and carry a regression test that fails without the fix. Issues marked ACKNOWLEDGED are accepted by design, with the rationale and any operational mitigations documented.

Issues Found

Critical	High	Medium	Low	Info
0	1	2	5	10

Issues Not Fixed and Not Acknowledged

Critical	High	Medium	Low	Info
0	0	0	0	0

Summary of Findings

ID	Severity	Title	Status
H-01	High	Donated liquidity on an unstaked Aerodrome vault NFT makes the whole debt unliquidatable	Fixed
M-01	Medium	A partial liquidation can take more than its share of the collateral and leave bad debt	Fixed
M-02	Medium	Redemption pays the oracle price with no buffer, so redeemers keep the oracle's lag behind the market	Fixed
L-01	Low	A liquidator can move the pool price to earn the bonus on the owner's eUSD	Fixed

L-02	Low	A blocklisted owner makes the final full liquidation revert	Acknowledged
L-03	Low	The unused-mint burn in `addLiquidity` retires fees first, so fees become principal	Fixed
L-04	Low	A redemption removes a whole range before it knows it can repay nothing	Fixed
L-05	Low	A one-wei rounding shortfall lets a redemption remove part of a `minDebt` range and repay nothing	Fixed
I-01	Info	`sqrtPriceFromOracle` overflows for a low-decimal asset at a high price	Acknowledged
I-02	Info	Aerodrome `addLiquidity` measures tokens used by balance difference across a gauge restake	Acknowledged
I-03	Info	A pool asset equal to the DEX reward token turns emissions into collateral	Acknowledged
I-04	Info	`addLiquidity` and `reposition` have no slippage limit beyond the pool-price band	Acknowledged
I-05	Info	eUSD in a range counts at $1, so a single-sided eUSD range is leveraged on the asset	Acknowledged
I-06	Info	A partial liquidation of a position below 100% can lower its ratio	Acknowledged
I-07	Info	A liquidator earns no bonus when the pool trades above the range	Acknowledged
I-08	Info	The operator mint pause does not stop redemption	Acknowledged

I-09	Info	`closePrincipalOnly` settles fees in asset at the bare oracle price	Acknowledged
I-10	Info	The unused-mint burn in `addLiquidity` emits no event	Fixed

Status of Earlier Findings

- No earlier report covers the EUSDLiquidityManager. The 01-11-2026 report's acknowledged item on a blocklisted owner's liquidation refund in `EUSDManager` has the same cause as L-02 here.

Properties Attacked and Held

EUSDLiquidityManager:

- eUSD supply equals `totalDebt`, which equals the sum of position debts, on every path (mint, repay, close, both liquidation kinds, redemption, the unused-mint burn and fee accrual).
- Every position's debt is zero or at least `minDebt` after every call.
- `feesAccrued` never exceeds `debt`.
- Valuation at the oracle price is a lower bound for the range at any pool price, so moving the pool can only lower a position's value, never raise it.
- Minted eUSD goes only into the position's vault, never to a user wallet.
- After the M-01 fix, a partial liquidation of a position at or above 100% never lowers its ratio.
- After the L-01 fix, the liquidator's take is `cleared × bonus` whatever the pool split.
- Redemption: the payout never exceeds the eUSD burned (rounding is against the redeemer), never exceeds the vault's asset, and raises the owner's ratio. Pushing the pool down to make a position eligible costs the attacker more than the payout. Naming an owner twice, naming owners with no debt, and redeeming one wei are all safe.
- The staked and unstaked fee indexes stay consistent across stake switches.

FablesAdapter / AerodromeAdapter:

- Only the manager can move principal; the owner can only sweep tokens other than the asset and eUSD.
- After the H-01 fix, a full removal burns the Aerodrome NFT's real liquidity, so donated liquidity cannot block a close or a liquidation.
- The Fables pool price read from the PoolManager storage slot matches the live contract.

Issue H-01: Donated liquidity on an unstaked Aerodrome vault NFT makes the whole debt unliquidatable RESOLVED

Severity: High

Vulnerability Detail

The Robinhood Chain Aerodrome position manager restricts `increaseLiquidity` only while the NFT is in the gauge. Anyone can therefore add liquidity to an unstaked vault NFT. The vault tracked only the liquidity it added itself, and a full removal removed only that amount, so `burn` reverted because liquidity remained.

The owner could then shrink the tracked liquidity to 1 with `removeLiquidity`. Once the range has filled to all asset, this does not change the debt. From then on, every liquidation computes `share = ceil(1 × cleared / debt) = 1 = held` and takes the full-removal path, which reverts.

Example: an eUSD-only range sits below the price with 3,000 eUSD of debt. The price falls through the range, the owner removes `held − 1`, and someone donates dust liquidity. With the oracle far below the range (ratio 49%), `liquidate(max)`, `liquidate(debt / 2)` and a small liquidation all revert.

Impact

The owner holds a free put on the asset: the debt can never be liquidated, so the protocol carries the bad debt, bounded only by the debt ceiling. A third party can also block any unstaked position's `close` and `reposition`.

Resolution

A full removal now reads the NFT's actual liquidity from `positions(id)` and removes all of it. Donated tokens land in the vault as idle balance. Regression tests: `test_donatedLiquidity_closeStillBurns`, `test_donatedLiquidity_heldOne_liquidationsWork` and the fork test `test_fork_aero_donatedLiquidity_liquidationsWork`. The mock position manager no longer gates `increaseLiquidity`.

Issue M-01: A partial liquidation can take more than its share of the collateral and leave bad debt RESOLVED

Severity: Medium

Vulnerability Detail

`_payLiquidator` capped the asset seized at the vault's whole asset balance, not at the share of the position being cleared.

Example with a 5% bonus: debt is 1,000,000 and the vault's asset is worth 1,020,000. A partial liquidation of 971,000 takes all of the asset and leaves 28,600 of debt with no collateral. A full liquidation of the same position leaves no bad debt.

Impact

A liquidator can turn a solvent position (ratio between 100% and 100% + bonus) into bad debt.

Resolution

A partial liquidation now takes at most `value × cleared / debt` in total (vault eUSD burned plus asset seized), the same pro-rata cap `EUSDManager` uses. Regression test: `test_liquidate_partial_belowOnePlusBonus_paysOnlyShareOfValue`.

Issue M-02: Redemption pays the oracle price with no buffer, so redeemers keep the oracle's lag behind the market RESOLVED

Severity: Medium

Vulnerability Detail

A redeemer burned eUSD and received asset at the oracle price with no fee. A redemption-only price age check was meant to keep that price current. The oracle reports any Chainlink answer younger than its fresh window as current (`block.timestamp`), and Chainlink stock feeds update on a deviation threshold. So the check did not limit how far the price could trail the market.

Example with an illustrative 0.5% deviation threshold: the feed shows 100.00 and the market trades at 100.45. A redeemer sells asset into the pool for eUSD, redeems 10,000 eUSD for 100 units of asset, and sells them for 10,045. Positions whose range sits above the price (all asset) are always eligible, so no sell-off or depeg is needed.

Impact

Owners of eligible positions lose the oracle–market gap on every redemption, up to the feed's deviation threshold during market hours and more when off-hours prices are held inside their band.

Resolution

Redemption now pays `repaid × (1 − redeemFeeBps) / price`. The fee stays in the owner's vault and is set by ADMIN; it should be at least the feed's deviation threshold. The redemption-only price age was removed, and redemption uses `mintPriceMaxAge`. A zero `redeemMinAssetShareBps` turns redemption off. Regression test: `test_redeem_feeStaysWithOwner`.

Issue L-01: A liquidator can move the pool price to earn the bonus on the owner's eUSD RESOLVED

Severity: Low

Vulnerability Detail

Liquidation removes the cleared share of the range at the live pool price, with no band (by design, so a lagging pool cannot block liquidations). The bonus was paid on `repaid = cleared − vault eUSD`. Pushing the pool below the range turns the removed eUSD into asset, which raises `repaid` and the asset seized.

Impact

The owner loses up to `bonus × the removed share's eUSD`, less the attacker's swap costs.

Resolution

The bonus is now paid on `cleared`: `seized = (repaid + cleared × bonus) / price`. The liquidator's take no longer depends on the pool split. Regression test: `test_liquidate_bonusOnCleared_sameWhateverThePoolSplit`.

Issue L-02: A blocklisted owner makes the final full liquidation revert ACKNOWLEDGED

Severity: Low

Vulnerability Detail

A full liquidation sends the asset refund to the owner in the liquidator's call. If the asset token blocks the owner, the transfer reverts. The last `minDebt` of a position can only be cleared by a full liquidation.

Impact

At most about `minDebt` per position stays unliquidatable while the owner is blocked.

Resolution

Acknowledged, consistent with `EUSDManager`. No listed asset is known to block holders.

Issue L-03: The unused-mint burn in `addLiquidity` retires fees first, so fees become principal RESOLVED

Severity: Low

Vulnerability Detail

`addLiquidity` mints up to `eusdMax` into the vault and burns what the range did not use. The burn went through `_reduceDebt`, which retires accrued fees first. The owner's unpaid fees were cleared and the same amount stayed as principal.

Impact

Fees turned into principal that earns further fees, and `closePrincipalOnly` could no longer settle them from asset.

Resolution

The unused-mint burn now reduces principal only and keeps the `minDebt` rule. Regression tests: `test_add_unusedMint_keepsFeesAccrued`, `test_add_unusedMint_keepsMinDebt`.

Issue L-04: A redemption removes a whole range before it knows it can repay nothing RESOLVED

Severity: Low

Vulnerability Detail

`_redeemFrom` set `repaid` from the request, removed range liquidity to free that much asset, and only then clamped `repaid` again for `minDebt`. When the vault's asset was worth less than the request and the remaining gap was below `minDebt`, the second clamp returned zero or `debt − minDebt`.

Example (illustrative `minDebt` of 100): a position owes 100 and its range holds asset worth 97. A redemption of 101 against it and a second owner removes all of its range, repays 0 for it, and redeems from the second owner, so the call does not revert.

Impact

Anyone can pull a near-`minDebt` position's range out of the pool and gauge for the cost of gas. The owner keeps the value but loses swap fees and emissions until they add liquidity again.

Resolution

`repaid` is now capped by the eUSD the vault's whole asset covers before any range is removed, and a position that can repay nothing is skipped untouched. Regression test: `test_redeem_belowDebt_atMinDebt_keepsRange`.

Issue L-05: A one-wei rounding shortfall lets a redemption remove part of a `minDebt` range and repay nothing RESOLVED

Severity: Low

Vulnerability Detail

`_redeemCheck` sizes the range's asset with `holdings(poolPrice())`. The pool's sqrt price is converted to an 18-decimal price and back, and both steps round down. When the asset is `currency0` and the pool is inside the range, the estimate can exceed what the pool returns by one wei. `_freeAsset` then removes a share of the range and gets one wei less than `seized`. The fallback `_redeemable(debt, _redeemCover(idle))` sees a gap below `minDebt` and, for a position at exactly `minDebt`, returns zero.

Exact-integer replay with test-fixture parameters: 5.7% of the range is removed, nothing is burned, nothing is paid. The redeemer names a second owner so the call does not revert.

Impact

Grief only. The removed share stays in the vault as idle asset and idle eUSD, so the owner loses nothing but the fee income on that share. It happens once per position: a second call redeems normally.

Resolution

Fixed in `aba9e7e`. After `_freeAsset`, `_redeemFrom` now caps `seized` at the asset that is idle and keeps `repaid`, so the redeemer absorbs the rounding dust (at most a few wei). The `redeem` NatSpec says so. Test: `testFuzz_redeem_atMinDebt_poolRoundingStillRepays` puts the pool just above the range's lower tick, redeems a position at `minDebt`, and checks it is fully repaid and the payout is short by at most 2 wei. The test fails on `5a834c0`.

Issue I-01: `sqrtPriceFromOracle` overflows for a low-decimal asset at a high price ACKNOWLEDGED

Severity: Info

Vulnerability Detail

`Math.mulDiv(price, Q192, assetScale)` overflows when the price per raw asset unit is at least 2^64: above about $18.4 for a 0-decimal asset, $1,844 for 2 decimals and $18.4M for 6 decimals.

Impact

Valuation of that pool reverts. 18-decimal assets are never affected.

Resolution

Acknowledged. Do not list assets below 6 decimals.

Issue I-02: Aerodrome `addLiquidity` measures tokens used by balance difference across a gauge restake ACKNOWLEDGED

Severity: **Info**

Vulnerability Detail

For a staked NFT, `addLiquidity` measures the asset used as a balance difference that spans `gauge.withdraw` and `gauge.deposit`. If the gauge paid the pool asset into the vault, the subtraction would underflow.

Impact

Only a pool whose asset is the gauge's reward token is affected, and only the caller's call reverts.

Resolution

Acknowledged. No such pool is planned.

Issue I-03: A pool asset equal to the DEX reward token turns emissions into collateral ACKNOWLEDGED

Severity: Info

Vulnerability Detail

In a pool whose asset is FABLES (or the Aerodrome reward token), claimed emissions are the protected asset. `FablesAdapter.claimEmissions` then reverts in `_sweep` with `ProtectedToken`.

Impact

The emissions stay in the vault as collateral and leave on close.

Resolution

Acknowledged. No such pool is planned.

Issue I-04: `addLiquidity` and `reposition` have no slippage limit beyond the pool-price band ACKNOWLEDGED

Severity: Info

Vulnerability Detail

The only guard on the pool price at add and reposition time is the manager's band around the oracle price, plus the caller's maximum amounts.

Impact

A caller can get a worse fill inside the band. Someone holding the pool out of band can block an add for one block.

Resolution

Acknowledged. The band is the intended guard.

Issue I-05: eUSD in a range counts at $1, so a single-sided eUSD range is leveraged on the asset ACKNOWLEDGED

Severity: **Info**

Vulnerability Detail

A range below the price holds only eUSD, valued at $1. As the price falls through it, the range fills with asset bought with that eUSD, so the position is long the asset with leverage of about `1 / (MCR − 1)`.

Impact

A parameter note for low-MCR managers, not a code defect.

Resolution

Acknowledged.

Issue I-06: A partial liquidation of a position below 100% can lower its ratio ACKNOWLEDGED

Severity: Info

Vulnerability Detail

The pro-rata cap from M-01 bounds the asset seized but not the vault eUSD burned against the cleared debt. For a position already below 100% that holds idle eUSD, the burn can exceed the cap.

Example: debt 100, idle eUSD 90, idle asset worth 5 (ratio 95%). `liquidate(50)` burns 50 vault eUSD and seizes nothing; `_burnIdle` burns the other 40. Debt is 10, value is 5, ratio 50%.

Impact

None in value: owner equity, bad debt and the supply invariant are unchanged, and the liquidator gains nothing. The interface's statement that the remaining ratio never falls holds only at or above 100%.

Resolution

Acknowledged.

Issue I-07: A liquidator earns no bonus when the pool trades above the range ACKNOWLEDGED

Severity: Info

Vulnerability Detail

When the pool is above the range, the removed share is all eUSD, `repaid` is zero and the asset cap leaves no bonus. The debt is still cleared one for one from the vault's eUSD.

Impact

Liquidators have no incentive in this state, but solvency improves with every such liquidation.

Resolution

Acknowledged.

Issue I-08: The operator mint pause does not stop redemption ACKNOWLEDGED

Severity: Info

Vulnerability Detail

`withdraw` checks `mintPaused` before it releases asset at the oracle price; `redeem` does not. Only ADMIN can stop redemption, by setting `redeemMinAssetShareBps` to 0.

Impact

During an oracle incident, redemptions continue at a suspect price (within the oracle's band) until ADMIN acts. Every redemption still raises the owner's ratio.

Resolution

Acknowledged. The mint pause is for minting; ADMIN stops redemption with `setRedeemParams(0, feeBps)`. Every redemption raises the owner's ratio, so a short delay does not put a position at risk.

Issue I-09: `closePrincipalOnly` settles fees in asset at the bare oracle price ACKNOWLEDGED

Severity: Info

Vulnerability Detail

`closePrincipalOnly` pays the accrued fees in asset at the oracle price, rounded up by one unit. The owner chooses per call between paying fees in eUSD (`close`) and in asset, so they can pick asset when the oracle is above the market.

Impact

The treasury receives asset worth up to `feesAccrued × oracle gap` less than the fees. Bounded by the position's own fees.

Resolution

Acknowledged. The gap is bounded by the position's own fees and the oracle's band, and the path exists so an owner can close without buying eUSD.

Issue I-10: The unused-mint burn in `addLiquidity` emits no event RESOLVED

Severity: Info

Vulnerability Detail

`_add` emits `EUSDMinted` with the debt before the unused burn, and `_burnUnminted` lowers the debt without an event. `_burnIdle` emits `DebtRepaid` for the same kind of burn.

Impact

An indexer that rebuilds debt from events overcounts it. On-chain state is correct.

Resolution

Fixed in `aba9e7e`. `_burnUnminted` emits `DebtRepaid(poolId, owner, vault, amount, newDebt)`, the same event `_burnIdle` emits. Test: `test_add_unusedMint_emitsDebtRepaid` (fails on `5a834c0`).